PORTFOLIO PROJECT 01 · DATABASE RELEASE SAFETY

sql-patch-guard

A linter that reads Oracle & PostgreSQL patch scripts and stops the statements that silently destroy production data – before anyone runs them.

14/14 tests passing

Python 3.10+ · argparse · regex parsing · pytest · GitHub Actions · Oracle · PostgreSQL

Sallehuddin Mohd Zakinuddin · Senior Application Developer · Portfolio project on GitHub

1 Problem Statement

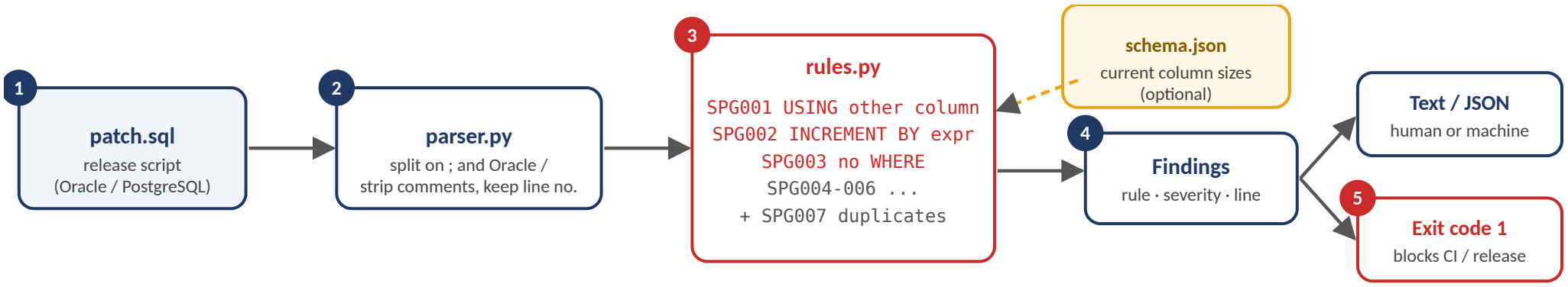
Database patches for a multi-environment system are written by several people, copied between releases and run by an operations team under time pressure. A single bad line can **silently destroy production data** – the database happily accepts it and no error appears.

Real example from my work: a release patch with seven copy-pasted `ALTER ... USING area::numeric` lines that would have overwritten seven columns with the value of a different column. I caught it by reading line by line.

2 Objectives

1. Turn manual, line-by-line patch review into an **automated, repeatable check**.
2. Catch the mistakes that actually happen in release patches (not generic SQL style).
3. Point to the **exact file and line** with a plain-English reason and a suggested fix.
4. Return a non-zero exit code so it can **block a CI pipeline or release script**.
5. Zero dependencies – runs anywhere Python 3.10+ runs.

3 System Architecture – how a patch flows through the tool



① **Input.** The `.sql` files exactly as handed to the DBA.

② **Parse.** Split statements; ignore `;` inside strings/comments; support Oracle `/`.

③ **Rules.** Each rule is a small `@rule` function – a new rule is ~10 lines.

④ **Findings.** Rule, severity, line number, reason and the statement.

⑤ **Gate.** Exit code `1` on errors stops the release pipeline.

4 Anatomy of the Bug it was built for

BEFORE (live data)

id	area	area_surveyed
1	120.5000	118.2500
2	88.0000	90.1250
3	301.7500	295.0000

AFTER running the patch

id	area	area_surveyed
1	120.5000	**120.50000**
2	88.0000	**88.00000**
3	301.7500	**301.75000**

```
ALTER TABLE land_parcel ALTER COLUMN area_surveyed
TYPE numeric(16,5) USING area::numeric(16,5);
```

Intent: widen the column. Effect: copy `area` into every row. No error is raised.

Why it is dangerous: PostgreSQL's `USING` clause defines how to compute the new value of the column. Pointing it at another column is valid SQL, so the statement succeeds. Because `area` and `area_surveyed` are already the same type, the correct fix is simply to **remove the USING clause**.

5 Rules Implemented

Code	Level	Catches
SPG001	ERROR	`USING` reads a different column → silent overwrite
SPG002	ERROR	`INCREMENT BY 500 - 20` → ORA-01722
SPG003	ERROR	`UPDATE` / `DELETE` with no `WHERE`
SPG004	WARN	`DROP` / `TRUNCATE` (needs backup plan)
SPG005	WARN	`NOT NULL` column added without `DEFAULT`
SPG006	WARN	Column narrowed vs. current schema
SPG007	WARN	Duplicate statement (copy-paste slip)

ERROR = must fix before running. WARN = allowed, but a human should confirm it is intended.

6 Results

7 rules

14 tests passing

4 / 7 errors / warnings found in sample patch

7 Sample Output

```
$ sql-patch-guard examples/risky_patch.sql --schema examples/schema_snapshot.json
risky_patch.sql:7:  ERROR SPG001 column 'area_surveyed' is rewritten from 'area' - drop the USING clause
risky_patch.sql:11: WARN  SPG006 application.remarks length 4000 -> 100
risky_patch.sql:14: WARN  SPG005 NOT NULL column without DEFAULT - existing rows will reject it
risky_patch.sql:17: ERROR SPG003 statement affects every row in the table (no WHERE clause)
risky_patch.sql:20: ERROR SPG002 INCREMENT BY '50213 - 49870' is not a literal (ORA-01722)
1 file(s) checked: 4 error(s), 7 warning(s)                    exit code = 1
```

The corrected `safe_patch.sql` passes with 0 errors – CI lints it on every push ("dogfooding"). Also lints the Flyway migration of land-registry-api in its CI.

Try it: `pip install -e ".[dev]"` then `sql-patch-guard --list-rules` · add `--format json` for machine output.

8 Key Concepts Learned

- **Valid ≠ safe.** The most dangerous SQL is the kind the database accepts without complaint.
- **Exit codes** are how tools talk to pipelines: 0 = pass, non-zero = stop.
- **Tokenising before matching**: strip comments and strings first, or a `;` inside text breaks everything.
- **Test both sides**: every rule has "should flag" and "should NOT flag" tests.

9 Limitations & Future Work

- Regex-based, not a full SQL grammar – tuned for release DDL, not complex PL/SQL bodies.
- Next: read the schema snapshot straight from the database instead of JSON.

10 Interview Talking Points

Q: Why not just review carefully?
People miss the 6th identical-looking line late at night. A tool never gets tired.

Q: How do you avoid false positives?
SPG001 only fires if the USING expression references another column *and not* the target – `USING trim(lot_no)::int` on `lot_no` is allowed.

Portfolio project · fictional data · MIT licence